

PROJECT MANAGEMENT IS

Sprint 1 report

Project: AI Student Request Routing and Support System

Increment: Student intake, AI routing, and notification receipts

Duration: Two weeks, as required by the lab sheet

Source scope: US1 and US2, 8 total source effort points

Readiness: Implemented increment; see executed verification evidence

Human acceptance: Not recorded; actual team/teacher acceptance must be recorded after the defence

Sprint goal

Enable an authenticated student to submit a request, reach the relevant department through a supervised classifier, and receive a confirmation in one working portal.

Planning and selected backlog

Story	Source effort	Owner role	Dependency	Completion evidence
US1. Classification and routing	5	AI/Backend Developer and QA Tester	Authentication, departments, database	Classifier, routing service, routed request test
US2. Confirmation and notifications	3	Frontend Developer	US1 and notification persistence	Receipt content/timing /duplication tests

Authentication, request CRUD, schema setup, privacy controls, and a consistent interface are enabling tasks. Their effort is tracked in the task backlog and is not added to the supplied story estimates. Owners use the roles supplied in the workbook because personal team assignments were not provided.

The workbook gives US1 a deadline of 15.11.2026 and US2 30.11.2026. These source dates are preserved. They do not define a single two-week sprint and are not presented as actual completion/defence dates. The course calendar and Moodle defence dates take precedence for submission scheduling.

Increment delivered

- SDU-only nine-digit student email signup, derived course, secure login/logout and sessions.
- An eight-entity SQLite schema, department configuration, and persistent request CRUD.
- Local Naive Bayes trained on 96 examples; server-only OpenAI routing is available as a configurable extension.
- Four department destinations: IT, finance, academic advising, and health services.
- Safe low-confidence escalation storage rather than guessing a department.
- Atomic request, event history, and in-app notification creation.
- Confirmation with request reference, department, and estimated initial response.

- Notification history/read markers with no duplicate notice on refresh.
- Responsive student interface with validation, errors, empty states, and request details.

Run `python run.py --demo --ai local --sprint 1 --database instance/sprint1.sqlite3` to demonstrate the Sprint 1 view. Support/admin routing tools are disabled in this mode. It uses the shared schema and safety path; the interactive review workflow is delivered in Sprint 2.

Acceptance and testing

The source's ten-request target is checked end-to-end through authenticated request creation. The independent 40-example synthetic holdout is evaluated separately. Notification checks verify the reference, department, response estimate, delivery timestamp, and refresh behavior. SDU signup rejects other domains and forged role/course values. The 24-prefix example is Year 3 in 2026-2027. Ownership and CSRF checks protect requests; mailbox ownership is not verified.

`docs/evidence/verification.json`, `test_results.txt`, and `model_evaluation.json` record actual execution results. Performance figures are environment-specific. The small dataset measures local mode only. OpenAI transport/failure tests are simulated; live accuracy is pending a configured key.

Sprint review / demo

1. Start Sprint 1, open the portal, sign in as the demo student.
2. Submit the Wi-Fi example from the demo guide.
3. Show the IT route and notification with response estimate.
4. Refresh notification history and show that the receipt is not duplicated.
5. Submit an ambiguous example and show safe staff-review placement.
6. Show the source backlog, Sprint 1 task board, and executed acceptance results.

This is a prepared review sequence. It is not evidence that a Product Owner attended or approved a meeting.

Technical retrospective

What worked: a local supervised model removes API-key/network dependencies during defence; separate modules make routing and persistence testable; one transaction keeps receipt and request state consistent.

Challenges: a tiny model can be overconfident about unfamiliar input. Evidence guards and a configurable threshold provide an explicit abstention path. The source backlog dates do not align with the lab's two-week duration, so source dates and course scheduling are documented separately.

Next improvement: add configurable human review and an accessible staff queue in Sprint 2; validate with broader authorized data later. Team ceremony feedback and individual participation must be added from real meeting records.

Progress tracking

`backlog.csv` lists technical tasks and their current evidence. `burndown.csv` contains an ideal 10-working-day plan plus a single build snapshot. It does not invent historical daily completions or accepted team velocity. The offline board is `docs/kanban.html`.